CHAPTER 1 · SUB-CHAPTER 1.1

Java Primitives & Type System

Primitive Types · Wrapper Classes · Autoboxing · Type Conversions
Integer Cache · IEEE 754 Floating Point · String Interning
equals/hashCode Contract · Pass-by-Value · Advanced JVM Representation

~20 Pages	50+ Q&A	Adv. Questions	Interview Ready

■ Purple [ADV] questions indicate advanced / senior-level depth

Section 1 — The 8 Primitive Types

1.1 Complete Primitives Reference

Primitive	Bits	Min	Max	Wrapper	Default	Literal Suffix
byte	8	-128	127	Byte	0	—
short	16	-32,768	32,767	Short	0	—
int	32	-2,147,483,648 (–2^{31})	2,147,483,647 (2^{31}–1)	Integer	0	—
long	64	-9,223,372,036,854,775,808	9,223,372,036,854,775,807	Long	0L	L or l
float	32	~1.4e-45	~3.4e+38 (7 sig dig)	Float	0.0f	F or f
double	64	~4.9e-324	~1.8e+308 (15 sig dig)	Double	0.0d	D or d (opt)
char	16	0 (\u0000)	65,535 (\uFFFF)	Character	\u0000	— (Unicode)
boolean	1*	—	—	Boolean	false	—

* boolean size is JVM-implementation-defined; in arrays, JVM typically uses 1 byte per element.

1.2 Numeric Literals & Underscore Syntax (Java 7+)

```
// Decimal, Hex, Octal, Binary
int dec = 1_000_000; // underscores allowed between digits (Java 7+)
int hex = 0xFF_EC_D1_B4; // 0x prefix
int oct = 0777; // 0 prefix (rarely used)
int bin = 0b1010_1010; // 0b prefix (Java 7+)
long big = 9_223_372_036L; // must have L suffix for long literal
// Floating-point literals
float f1 = 3.14f; // must have f/F — else compiler error (double -> float)
double d1 = 3.14; // default type of floating point literal
double d2 = 6.02e23; // scientific notation
double d3 = 0x1.fp10; // hexadecimal floating-point (rare)
// char literals
char c1 = 'A'; // single character
char c2 = 'A'; // Unicode escape (also 'A')
char c3 = 65; // integer value within 0-65535
char c4 = '
'; // escape sequences: \t \r \\ \'
```

1.3 Default Values (Fields vs Local Variables)

■■ TRAP: Local variables MUST be explicitly initialised before use — the compiler will report an error. Only instance and class (static) fields receive default values automatically.

```
class Demo {
int instanceInt; // default 0
double instanceDbl; // default 0.0
boolean instanceBool; // default false
String instanceStr; // default null (reference type)
void method() {
int localInt; // NO default — MUST initialise
// System.out.println(localInt); // compile error: variable not initialised
localInt = 5; // OK after explicit assignment
}
}
```

Section 2 — Type Widening, Narrowing & Promotion

2.1 Widening (Implicit) Conversion Hierarchy

```
// Automatic widening — no data loss for integers; possible precision loss for float/double
// byte -> short -> int -> long -> float -> double
// char -> int (and then follows same path)
```

```
byte b = 42;
short s = b; // widening: OK
int i = s; // widening: OK
long l = i; // widening: OK
float f = l; // widening: OK (but may lose precision for large longs!)
double d = f; // widening: OK
// PRECISION LOSS TRAP:
long bigLong = 123_456_789_012_345L;
float asFloat = bigLong; // 1.2345679E14 — precision lost!
double asDouble= bigLong; // 1.2345678901234E14 — also some loss
// char is an unsigned 16-bit type — widens to int as unsigned value
char ch = 'A'; // 65
int n = ch; // 65 — no cast needed (char -> int is widening)
```

2.2 Narrowing (Explicit Cast) — Truncation & Bit Masking

```
// Narrowing requires explicit cast — data may be lost
int bigInt = 300;
byte narrow = (byte) bigInt; // 300 & 0xFF = 44 (bitwise truncation)
// How narrowing works: keep the low-order bits of the source value
// 300 in binary: 0000_0001_0010_1100
// Keep 8 bits: 0010_1100 = 44
double pi = 3.14159;
int intPi = (int) pi; // 3 — truncation toward zero (NOT rounding)
int neg = (int) -3.99; // -3 — also truncation toward zero
// char <-> int narrowing/widening
int code = 65;
char ch = (char) code; // 'A'
int back = ch; // 65 (widening — no cast)
// Classic trap: byte arithmetic result is int!
byte x = 10, y = 20;
// byte z = x + y; // COMPILE ERROR: int cannot be assigned to byte
byte z = (byte)(x + y); // OK — explicit narrowing cast
```

2.3 Numeric Promotion Rules

Operation	Rule	Example
byte/short/char arithmetic	Both operands promoted to int first	byte a=10; byte b=20; int c=a+b; // auto
Mixed int + long	int promoted to long	int i=5; long l=10L; long r=i+l;
Mixed int/long + float	Integral promoted to float	long l=9L; float r=l+3.14f;
Any + double	Other operand promoted to double	int i=3; double r=i+1.5;
Assignment compound ops	Implicit narrowing cast included	byte b=10; b+=5; // OK (b=(byte)(b+5))

Section 3 — Overflow, Underflow, IEEE 754 Floating Point

3.1 Integer Overflow (Silent Wrap-Around)

```
// Integer arithmetic silently wraps — NO exception thrown!
int maxInt = Integer.MAX_VALUE; // 2_147_483_647
int wrap = maxInt + 1; // -2_147_483_648 (wraps to MIN_VALUE!)
long maxLong = Long.MAX_VALUE;
long wrapL = maxLong + 1; // Long.MIN_VALUE
// Detecting overflow — Java 8+ Math.addExact / multiplyExact throw ArithmeticException
int safe = Math.addExact(maxInt, 1); // throws ArithmeticException: integer overflow
long mul = Math.multiplyExact(100_000L, 100_000L); // OK: 10_000_000_000
// Classic 2's complement: all primitives use 2's complement representation
// Negation: ~x + 1
// Integer.MIN_VALUE negated = Integer.MIN_VALUE! (no positive counterpart)
System.out.println(-Integer.MIN_VALUE == Integer.MIN_VALUE); // true — overflow!
```

3.2 IEEE 754 — float & double Special Values

```
// Special float/double values (no exception on division by zero for floats)
double posInf = 1.0 / 0.0; // Double.POSITIVE_INFINITY
double negInf = -1.0 / 0.0; // Double.NEGATIVE_INFINITY
double nan = 0.0 / 0.0; // Double.NaN (Not a Number)
// NaN is NOT equal to itself — most confusing Java gotcha!
System.out.println(nan == nan); // false ← NaN != NaN
System.out.println(Double.isNaN(nan)); // true ← correct check
System.out.println(nan != nan); // true
// Infinity arithmetic
System.out.println(posInf + 1); // Infinity
System.out.println(posInf - posInf); // NaN
System.out.println(posInf * -1); // -Infinity
System.out.println(1.0 / posInf); // 0.0
// Integer division by zero DOES throw ArithmeticException: / by zero
// int x = 5 / 0; // throws!
// Floating point precision — not exact!
System.out.println(0.1 + 0.2); // 0.30000000000000004 ← NOT 0.3
// Use BigDecimal for monetary calculations
BigDecimal result = new BigDecimal("0.1").add(new BigDecimal("0.2")); // 0.3 exact
// Comparing doubles — never use == for calculated values
double a = 0.1 + 0.2, b = 0.3;
System.out.println(Math.abs(a - b) < 1e-9); // true — epsilon comparison
```

3.3 Bit Representation Details

Type	Sign Bit	Exponent Bits	Mantissa Bits	Precision
float (IEEE 754 single)	1	8	23	~7 decimal digits
double (IEEE 754 double)	1	11	52	~15-16 decimal digits

```
// Inspect raw bits
int floatBits = Float.floatToRawIntBits(3.14f); // 32-bit IEEE 754 layout
long doubleBits = Double.doubleToRawLongBits(3.14); // 64-bit IEEE 754 layout
// Useful for: NaN-boxing, fast inverse sqrt (Quake trick), serialisation
// Special bit patterns
Float.intBitsToFloat(0x7f800000); // +Infinity
Float.intBitsToFloat(0xff800000); // -Infinity
Float.intBitsToFloat(0x7fc00000); // quiet NaN
```

Section 4 — Wrapper Classes, Autoboxing & Integer Cache

4.1 Autoboxing & Unboxing

```
// Autoboxing: compiler inserts Integer.valueOf(i)
int i = 42;
Integer wi = i; // autoboxing: Integer.valueOf(42)
// Unboxing: compiler inserts wi.intValue()
int j = wi; // unboxing: wi.intValue()
// Performance trap — boxing in tight loop
List<Integer> list = new ArrayList<>();
for (int x = 0; x < 1_000_000; x++) {
list.add(x); // autoboxes 1M ints → 1M Integer objects on heap
}
// Better for numeric computation: use int[] or IntStream
// NullPointerException from unboxing null
Integer nullInt = null;
int bad = nullInt; // NPE! unboxing invokes null.intValue()
// Always guard: int safe = (nullInt != null) ? nullInt : 0;
// Or: int safe = Objects.requireNonNullElse(nullInt, 0);
// Method overloading ambiguity
void method(int i) { System.out.println("int"); }
void method(Integer i) { System.out.println("Integer"); }
method(5); // prints "int" — compiler prefers widening over boxing
method((Integer)5); // prints "Integer" — explicit box
```

4.2 Integer Cache — The == Trap

■■ CRITICAL INTERVIEW TRAP: Integer.valueOf() caches Integer objects for values in [-128, 127]. Values outside this range create new objects, so == returns false even for logically equal values. ALWAYS use .equals() for Integer comparison.

```
// Cache range: -128 to 127 (JLS guaranteed minimum)
Integer a = 100; // Integer.valueOf(100) — from cache
Integer b = 100; // same cached object
System.out.println(a == b); // true (same object reference!)
System.out.println(a.equals(b)); // true
Integer c = 200; // Integer.valueOf(200) — new object (outside cache)
Integer d = 200; // another new object
System.out.println(c == d); // false ← TRAP!
System.out.println(c.equals(d)); // true ← CORRECT
// The cache is controlled by -XX:AutoBoxCacheMax=<n> JVM flag
// Default max is 127; can be raised but NOT lowered below 127
// Applying to other wrappers:
// Byte, Short, Integer, Long all cache [-128, 127]
// Character caches [0, 127] (■ to ■)
// Boolean: Boolean.TRUE and Boolean.FALSE are singletons (always cached)
// Float, Double: NO cache (infinite possible values)
// Long cache example (same rule)
Long la = 127L, lb = 127L;
System.out.println(la == lb); // true (cached)
Long lc = 128L, ld = 128L;
System.out.println(lc == ld); // false (not cached)
```

4.3 Wrapper Class Utility Methods

Wrapper	Key Utility Methods	Notes

Integer	parseInt(s), valueOf(s), toBinaryString(n), toHexString(n), bitCount(n), reverse(n), highestOneBit(n), numberOfLeadingZeros(n)	parseInt throws NumberFormatException; valueOf returns cached obj
Long	parseLong(s), toBinaryString(n), compareUnsigned(a,b), divideUnsigned(a,b)	Unsigned arithmetic ops added Java 8
Double	parseDouble(s), isNaN(d), isInfinite(d), compare(a,b), sum/min/max	compare() treats NaN == NaN (unlike ==)
Character	isDigit(c), isLetter(c), isUpperCase(c), toLowerCase(c), isWhitespace(c), getType(c), digit(c,radix)	Handles Unicode properly; isDigit('■') = true
Boolean	parseBoolean(s), TRUE, FALSE singletons	Only 'true' (case-insensitive) is true; everything else false

Section 5 — String, Immutability & equals/hashCode Contract

5.1 String Immutability & the String Pool

```
// String is immutable — final class, char[] value field is final
// All "modification" methods return a NEW String
// String pool (interned strings)
String a = "hello"; // literal → stored in pool (Metaspace, Java 8+)
String b = "hello"; // same pool entry → a == b is TRUE
String c = new String("hello"); // new heap object → c == a is FALSE
String d = c.intern(); // adds to pool; d == a is TRUE
// Compile-time constant folding
String s1 = "foo" + "bar"; // constant expression → "foobar" literal in pool
String s2 = "foobar";
System.out.println(s1 == s2); // true — same pool entry
// BUT runtime concatenation creates new object
String foo = "foo";
String bar = "bar";
String s3 = foo + bar; // runtime concat → new heap object
System.out.println(s3 == s2); // false
// StringBuilder vs StringBuffer vs + concatenation
// + in loop: O(n²) — creates new String each iteration
String r = "";
for (int i = 0; i < 1000; i++) r += i; // 1000 temp Strings created!
// StringBuilder: mutable, NOT thread-safe, O(n) amortized
StringBuilder sb = new StringBuilder(4096); // pre-size to avoid resizing
for (int i = 0; i < 1000; i++) sb.append(i);
String result = sb.toString(); // single final String
// StringBuffer: synchronized (legacy) — use only if truly needed
// Java 9+ JIT may optimise + chains — but explicit SB is clearest intent
// String.format vs formatted (Java 15+)
String msg1 = String.format("Hello %s, age %d", name, age);
String msg2 = "Hello %s, age %d".formatted(name, age); // same, cleaner
```

5.2 equals() & hashCode() Contract

Property	Definition	Violation Consequence
Reflexive	x.equals(x) must be true	Broken iterator.remove(), Set.contains()
Symmetric	x.equals(y) ↔ y.equals(x)	Inconsistent collection behavior
Transitive	x=y and y=z implies x=z	Breaks sorting, contains()
Consistent	Repeated calls same result (if fields unchanged)	Non-deterministic HashMap behavior
Non-null	x.equals(null) must return false (not NPE)	NullPointerException in collections
hashCode contract	equals(y) true → hashCode() == y.hashCode()	HashMap get() returns null for existing key

```
// Correct implementation
class Employee {
private final int id;
private final String name;
private final String dept;
@Override
public boolean equals(Object o) {
if (this == o) return true; // reflexive + fast path
if (!(o instanceof Employee e)) return false; // pattern matching (Java 16+)
return id == e.id && Objects.equals(name, e.name); // null-safe field compare
```

```
}
@Override
public int hashCode() {
return Objects.hash(id, name); // consistent with equals fields
}
}
// Common mistakes
// 1. Using @Override on wrong signature: equals(Employee e) — doesn't override Object!
// 2. Mutable fields in hashCode — objects change hash while in a HashSet → lost forever
// 3. Overriding equals only — objects logically equal but in different HashMap buckets
// 4. Including ALL fields in equals but fewer in hashCode — violates the contract
// Lombok @EqualsAndHashCode — auto-generates; use @EqualsAndHashCode(of={"id"})
// Records — auto-generate correct equals/hashCode from all components
```

Section 6 — Pass-by-Value: The Definitive Explanation

Java is ALWAYS pass-by-value. For primitives the value itself is copied. For objects a copy of the reference (pointer) is passed — the caller's variable is never reassigned by the callee, but the object's internal state CAN be mutated through the copied reference.

```
// Case 1: Primitive — copy of value
void doubleIt(int x) { x *= 2; } // only the local copy changes
int n = 5;
doubleIt(n);
System.out.println(n); // still 5
// Case 2: Object — copy of reference → mutation IS visible
void addElement(List<String> list) { list.add("added"); }
List<String> myList = new ArrayList<>();
addElement(myList);
System.out.println(myList.size()); // 1 — mutation visible (same underlying object)
// Case 3: Reassignment — caller's reference NOT changed
void replace(List<String> list) { list = new ArrayList<>(); } // local ref rebound
List<String> myList2 = new ArrayList<>(List.of("x"));
replace(myList2);
System.out.println(myList2.size()); // still 1 — caller's ref unchanged
// Analogy: passing a house key (reference copy):
// You can unlock the house and move furniture (mutate object) — visible to owner
// But making a new key doesn't change owner's key (reassignment not visible)
// Swap pattern — doesn't work in Java (unlike C++ refs)
void swap(int a, int b) { int t=a; a=b; b=t; } // DOES NOTHING to caller
// Workaround: return array/pair, or use AtomicInteger[]
```

Section 7 — Bitwise Operations & Bit Tricks

Operator	Name	Example	Result	Notes
&	AND	5 & 3	1	Both bits 1
|	OR	5 | 3	7	Either bit 1
^	XOR	5 ^ 3	6	Different bits
~	Complement	~5	-6	Flips all bits (2's complement)
<<	Left shift	5 << 1	10	Multiply by 2; fills right with 0
>>	Signed right shift	–8 >> 1	-4	Divide by 2; fills left with sign bit
>>>	Unsigned right shift	–1 >>> 1	2147483647	Fills left with 0 (ignores sign)

```
// Practical bit tricks used in JDK source
// HashMap index calculation (capacity is always power of 2)
int index = hash & (capacity - 1); // faster than hash % capacity
// Check if power of 2
boolean isPow2 = (n > 0) && ((n & (n - 1)) == 0);
// Next power of 2 (used in HashMap resize)
int nextPow2(int n) {
n--;
n |= n >> 1; n |= n >> 2; n |= n >> 4; n |= n >> 8; n |= n >> 16;
return n + 1;
}
// Set / clear / toggle a specific bit
int setBit = val | (1 << pos); // set bit at pos
int clearBit = val & ~(1 << pos); // clear bit at pos
int toggleBit = val ^ (1 << pos); // toggle bit at pos
```

```
boolean isSet = (val & (1 << pos)) != 0; // test bit
// Integer.bitCount — count set bits (Brian Kernighan / intrinsic)
int bits = Integer.bitCount(0b1010_1111); // 6
// Swap without temp variable using XOR
a ^= b; b ^= a; a ^= b; // works but never do this — unreadable
```

Interview Q&A; — 52 Questions (Q = Standard, [ADV] = Advanced)

Purple [ADV] questions target senior/principal-level interviews. Standard questions are suitable for mid-level Java engineers.

Part 1 — Primitives & Data Types

Q1. What are the 8 Java primitive types and their sizes?

byte (8-bit), short (16-bit), int (32-bit), long (64-bit), float (32-bit IEEE 754), double (64-bit IEEE 754), char (16-bit unsigned Unicode), boolean (JVM-defined, logically 1-bit). All integers use 2's complement; float/double use IEEE 754. There is no unsigned integer type in core Java (Java 8 added unsigned helpers in Integer/Long).

Q2. What are the default values for primitive fields in a class?

byte/short/int/long → 0, float → 0.0f, double → 0.0d, char → '\u0000' (null character), boolean → false. Local variables have NO default — using them uninitialized is a compile error. Reference types (including String) default to null.

Q3. Why can't you use primitives as generic type parameters?

Generics use type erasure and operate on reference types (Object at runtime). Primitives are not objects; they lack class metadata, inheritance, and cannot be stored as Object. Use wrapper classes instead: List not List. Java 23+ Project Valhalla's 'primitive types' aim to eventually allow value types in generics.

Q4. What is widening conversion? Give an example where precision is lost despite widening.

Widening converts a smaller type to a larger one implicitly (byte→short→int→long→float→double). Precision loss: long to float. A long holds 19+ significant decimal digits but float has only ~7. Example: long l=123_456_789_012_345L; float f=l; prints 1.2345679E14 — digits are lost.

Q5. Explain what happens when you cast (byte) 300.

300 in binary (32-bit int): 0000_0001_0010_1100. Narrowing to byte keeps the low 8 bits: 0010_1100 = 44. The upper 24 bits are discarded. Result is 44. Java uses 2's complement so if the kept 8 bits have the high bit set, the result is negative (e.g., (byte)200 = -56).

Q6. What is numeric promotion in Java?

Before arithmetic on byte, short, or char, Java promotes operands to int (minimum). If either operand is long, both become long. float/double similarly dominate. This is why 'byte a=10, b=20; byte c=a+b;' is a compile error — a+b produces int, not byte. Fix: byte c=(byte)(a+b). Compound operators (+=, *=) include an implicit narrowing cast.

Q7. Why does (int) 3.9 give 3, not 4?

Narrowing float/double to integer truncates toward zero — it is NOT rounding. (int)3.9 = 3, (int)-3.9 = -3. To round: (int)Math.round(3.9) = 4. This applies to all float→int casts: (int)Double.MAX_VALUE overflows to Integer.MAX_VALUE.

Q8. What is the difference between >> and >>> for a negative number?

>> is arithmetic (signed) right shift — fills vacated bits with the sign bit (1 for negative), preserving the sign. -8 >> 1 = -4. >>> is logical (unsigned) right shift — always fills with 0, making the result positive. -1 >>> 1 = 2147483647 (Integer.MAX_VALUE). Use >>> in hash functions to avoid sign propagation.

Q9. What is the char type and why is it unsigned?

char is a 16-bit unsigned type representing a Unicode code point in the Basic Multilingual Plane (U+0000 to U+FFFF). It's unsigned because Unicode code points are non-negative. It widens to int with no sign extension. char cannot represent supplementary Unicode characters (U+10000+); those require two chars (a surrogate pair) or a full int codePoint.

Q10. What is the range of the char type and what happens if you cast a negative int to char?

char ranges from '\u0000' (0) to '\uFFFF' (65,535). Casting a negative int: (char)(-1) = '\uFFFF' (65535). The int's bit pattern is reinterpreted as unsigned 16-bit. (char)(-2) = '\uFFFE' = 65534. This is why you should always check char ranges when building string processing utilities.

Part 2 — Autoboxing, Wrappers & Integer Cache

Q11. When does autoboxing occur?

Autoboxing is compiler sugar: 1) Assigning primitive to wrapper type. 2) Passing primitive to method expecting wrapper. 3) Adding primitive to a collection (List). 4) Mixed arithmetic with wrapper (Integer i=5; int j=i+3 — i is unboxed). The compiler inserts Type.valueOf() for boxing and type.typeValue() for unboxing.

Q12. What are the three most dangerous autoboxing pitfalls?

1) NullPointerException: Integer x=null; int y=x — compiler inserts x.intValue() on null. 2) == reference trap: Integer a=200, b=200; a==b is false (outside cache). 3) Performance: boxing in tight loops creates millions of short-lived objects causing GC pressure. Always profile before optimising, but be aware in hot paths.

Q13. Exactly which values does the Integer cache cover, and is it configurable?

JLS guarantees Integer.valueOf() caches [-128, 127]. The upper bound can be raised with -XX:AutoBoxCacheMax= JVM flag (HotSpot implementation detail, not JLS). The lower bound is always -128. Byte, Short, Long also cache [-128, 127]; Character caches [0, 127]; Boolean has TRUE/FALSE singletons; Float/Double have NO cache.

Q14. What does Boolean.parseBoolean("TRUE") return?

true. parseBoolean is case-insensitive: "true", "TRUE", "True" all return Boolean.TRUE. ANYTHING else ("yes", "1", "on", null, "") returns false — no exception. Contrast with Integer.parseInt which throws NumberFormatException for non-numeric input.

Q15. Explain why Collections cannot hold primitive types.

Java generics use type erasure to Object. At runtime a List is a List of Objects. Primitives are not Objects — they have no class hierarchy, no vtable, no object header. This forces boxing. Alternatives: IntStream (stream of int primitives), int[] arrays, or third-party libraries like Eclipse Collections / Trove that provide primitive collections.

Q16. What is the result of Integer.parseInt("2147483648")?

Throws NumberFormatException — 2147483648 is one more than Integer.MAX_VALUE (2147483647). It cannot fit in an int. Use Long.parseLong() for values up to 9,223,372,036,854,775,807. For arbitrary precision use new BigInteger("2147483648").

Q17. How does method overloading resolution work with autoboxing?

Compiler preference order: 1) Exact match (widening preferred over boxing). 2) Widening primitive. 3) Autoboxing. 4) Varargs. So method(int) is preferred over method(Integer) when passing an int literal. Ambiguity arises when multiple methods match after boxing — compiler reports error.

Q18. What does Integer.compare(a, b) do differently from a - b for sorting?

a - b is unsafe: for a=Integer.MIN_VALUE and b=1, the subtraction overflows to positive, incorrectly indicating a > b. Integer.compare(a, b) returns -1/0/1 without overflow: implementation is (a < b) ? -1 : ((a == b) ? 0 : 1). Always use Integer.compare in Comparators to avoid this classic comparison overflow bug.

Part 3 — String, Immutability, equals/hashCode

Q19. Why is String immutable in Java?

Four reasons: 1) Security — class names, file paths, DB credentials passed as Strings cannot be tampered with after validation. 2) String pool efficiency — multiple references can safely share one interned String object. 3) Thread-safety — immutable objects are inherently safe for concurrent use without synchronisation. 4) hashCode caching — String computes and caches its hashCode lazily on first call; immutability guarantees it never changes.

Q20. What happens under the hood when you concatenate Strings with +?

For string literals, the compiler constant-folds at compile time. For runtime concatenation, pre-Java-9 the compiler generated new StringBuilder().append(...).toString(). Java 9+ uses invokedynamic with StringConcatFactory — the JVM can choose the optimal strategy at runtime (typically more efficient than explicit StringBuilder). Inside loops + is still $O(n^2)$ if the loop creates a new chain each iteration; use explicit StringBuilder there.

Q21. What is the equals/hashCode contract and what breaks if you violate it?

Contract: if a.equals(b) then a.hashCode() must equal b.hashCode(). If you override equals but not hashCode (or use different fields), equal objects hash to different buckets. HashMap.get() finds the bucket via hashCode, then confirms with equals — it will miss your key entirely, returning null. HashSet.contains() will also fail to find your object.

Q22. What are the five mathematical properties of equals()?

Reflexive: x.equals(x)=true. Symmetric: x.equals(y)=y.equals(x). Transitive: x=y and y=z implies x=z. Consistent: same result across multiple calls (if state unchanged). Non-null: x.equals(null) must return false (never throw NPE). Violating any of these causes unpredictable behavior in Collections, sorting, and any equality-based algorithm.

Q23. How does String.intern() work and when should you use it?

intern() checks the string pool; if an equal String is already there it returns that reference, otherwise it adds this string and returns it. Use case: deduplicating large numbers of equal string objects (e.g., repeated column names parsed from CSV). Caution: internment pool lives in Metaspace; interning unbounded strings causes memory pressure. Java 8+ moved interned strings from PermGen to Metaspace, allowing GC of unused interned strings.

Q24. Why is String.substring() O(n) in Java 7u6+ but was O(1) before?

Before Java 7u6 substring shared the backing char[] with the original String (offset + count). This caused memory leaks: a tiny substring could keep a huge backing array alive. From Java 7u6 onward, substring copies the chars — O(n) time, O(n) space, but no memory leak. This is an important trade-off: use substring freely for correctness; only care about cost in very hot paths processing massive strings.

Part 4 — Pass-by-Value & Reference Semantics

Q25. Is Java pass-by-value or pass-by-reference? Explain precisely.

Always pass-by-value. For primitives the value is copied. For objects, the VALUE of the reference (the memory address) is copied. The callee gets its own copy of the reference — it can mutate the object (visible to caller), but reassigning the parameter only changes the local copy, not the caller's variable. Java has no pass-by-reference semantics at all.

Q26. How would you swap two integers in Java without a temp variable?

Approach 1 — XOR (works for distinct memory locations, unsafe if a==b reference): a^=b; b^=a; a^=b; Approach 2 — arithmetic (overflow risk): a=a+b; b=a-b; a=a-b; Approach 3 — return array: int[] swap(int a, int b){return new int[]{b,a};} Best practice: use a temp variable — it's clear, safe, and JIT eliminates it anyway. For objects, swap is simply impossible without a wrapper since you can't change the caller's reference.

Q27. Can a method modify a primitive argument's value as seen by the caller?

No. Primitives are copied. To allow a method to modify a value visible to the caller, you must: 1) Return the new value. 2) Wrap in an array: int[] ref = {5}; method(ref); 3) Use AtomicInteger. 4) Use a mutable holder object. Records and final fields are not options since they're immutable. Java simply does not have out/ref parameters like C#.

Part 5 — IEEE 754, Overflow & Precision

Q28. What does 0.1 + 0.2 == 0.3 evaluate to in Java?

false. 0.1 and 0.2 cannot be represented exactly in binary IEEE 754. Their sum is 0.30000000000000004. Use Math.abs(a-b) < epsilon for comparison, or BigDecimal for exact decimal arithmetic (new BigDecimal("0.1").add(new BigDecimal("0.2")) = 0.3 exactly).

Q29. What is the result of Integer.MAX_VALUE + 1?

Integer.MIN_VALUE (-2147483648). Java integer arithmetic silently wraps around — no exception. Use Math.addExact(Integer.MAX_VALUE, 1) to get ArithmeticException: integer overflow. Or use long for the calculation. This silent overflow is a major source of bugs in hash functions, loop bounds, and coordinate systems.

Q30. Is NaN == NaN true or false in Java?

false. NaN is never equal to anything, including itself — this is IEEE 754 specification. Use Double.isNaN(x) to test for NaN. In sorting: Double.compare(a,b) treats NaN as equal to itself and greater than all other values, which is consistent for Collections but contradicts ==. This is why Comparable and == can disagree for doubles.

Q31. What is the result of 1.0 / 0.0 and 0 / 0 in Java?

1.0/0.0 = Double.POSITIVE_INFINITY — no exception (IEEE 754 floating point rule). 0.0/0.0 = Double.NaN. BUT 0 / 0 (integer) throws ArithmeticException: / by zero. The key distinction: floating-point division by zero follows IEEE 754 (special values); integer division by zero throws an exception (no special integer infinity exists).

Q32. Why should you use BigDecimal for financial calculations?

float/double are binary floating-point — they cannot represent most decimal fractions exactly (0.1, 0.01, etc.). Rounding errors accumulate and produce incorrect monetary values. BigDecimal stores exact decimal representations. Always construct from String: new BigDecimal("0.1"), NOT new BigDecimal(0.1) which captures the double's imprecise value. Use RoundingMode.HALF_UP for standard monetary rounding.

Advanced Questions — Senior/Principal Level

The following questions probe JVM internals, subtle specification details, and real-world production scenarios. These appear in staff engineer and principal architect interviews.

Part 6 — JVM Memory & Runtime Representation

[ADV] Q33. How are primitives vs objects stored differently in JVM memory?

Primitives on the stack (local variables/parameters) are stored directly as raw bit patterns in the stack frame — no object header, no indirection. Heap-allocated primitives (fields of objects) are stored inline within the object's memory layout. Boxed wrappers (Integer, Long) are full heap objects with a 12-16 byte object header plus the value field. This is why primitive fields are faster to access and cause less GC pressure than wrapper fields.

[ADV] Q34. What is value type flattening and why does Project Valhalla pursue it?

Currently, generics work only with references — List is impossible, forcing boxing. Project Valhalla (previewing in Java 23+) introduces primitive classes / value types: immutable objects with no identity that can be stored inline (flattened) in arrays and generic containers without boxing overhead. int[] already has flattening; Valhalla extends this to custom types. This can reduce memory footprint and GC pressure for numeric-heavy workloads by 5-10x.

[ADV] Q35. Explain how String.hashCode() is implemented and why it uses 31 as the multiplier.

s.hashCode() = s[0]*31^(n-1) + s[1]*31^(n-2) + ... + s[n-1]. Result is cached in a private hash field (lazy init). 31 chosen because: it's a prime (reduces collisions), 31*i = (i<<5)-i (JIT can optimise to shift+subtract — was important before hardware multiply was fast). Note: hash=0 is ambiguous with 'not yet computed', but the empty string also hashes to 0, so a recomputation check uses hash==0 — harmless but a subtle JDK implementation detail.

[ADV] Q36. How does the JVM lay out object memory? What is object header overhead?

HotSpot object header: mark word (8 bytes — GC age, lock state, hash) + klass pointer (4 bytes with compressed refs enabled, 8 bytes without). Total: 12-16 bytes before any fields. Fields are then laid out with alignment padding. An Integer object = 12 (header) + 4 (int value) + 0 padding = 16 bytes. Compare to a raw int: 4 bytes. Primitive int is 4x smaller in memory. For arrays: 16 bytes header + length (4) + elements. Object arrays store references (4/8 bytes each).

[ADV] Q37. What is NaN-boxing and where is it used?

NaN-boxing encodes non-double values inside the bit patterns of IEEE 754 NaN values. A double has 2^52 - 2 possible NaN bit patterns; only one (canonical NaN) is typically used. JavaScript engines (V8, SpiderMonkey) use the remaining NaN patterns to store pointers and integer values in a single 64-bit slot — this is how dynamic languages achieve compact representation. Java doesn't use NaN-boxing directly, but understanding it shows depth in JVM and engine internals.

Part 7 — Subtleties & Gotchas

[ADV] Q38. What is the output of: System.out.println('a' + 'b')?

195. Char literals are promoted to int before arithmetic. 'a'=97, 'b'=98, sum=195. NOT "ab". To concatenate: System.out.println("" + 'a' + 'b') = "ab" (String concat). Or: System.out.println(Character.toString('a') + 'b'). This catches many developers off guard in coding interviews.

[ADV] Q39. What is the output of: Integer a = 1000; Integer b = 1000; System.out.println(a > b);

false. a > b triggers unboxing of both to int (1000 > 1000 = false). The key trap: > always unboxes — it cannot compare object references. So a>b, a=b, a<=b always unbox. Only == and != operate on references (and thus can give wrong results for cached/non-cached Integers). Use a.compareTo(b) or Integer.compare(a,b) for safe comparison.

[ADV] Q40. Is there any situation where -Integer.MIN_VALUE is positive?

No. -Integer.MIN_VALUE overflows back to Integer.MIN_VALUE. 2's complement: MIN_VALUE = 1000...0 (32 bits). Negation: flip bits → 0111...1 = MAX_VALUE; add 1 → 1000...0 = MIN_VALUE again. This means Math.abs(Integer.MIN_VALUE) = Integer.MIN_VALUE — another classic JDK gotcha. Math.absExact(Integer.MIN_VALUE) throws ArithmeticException (Java 15+).

[ADV] Q41. What are the compiler rules for final local variables and assignment?

A final local variable is a blank final — must be assigned exactly once before use; the compiler tracks all code paths. int: assigned on all paths through if/else/switch = OK. If any path skips assignment, compile error. Loops: compiler cannot determine iteration count, so final variables declared inside a loop body are re-initialised each iteration. Effectively final (Java 8+): not declared final but never reassigned — can be used in lambdas.

[ADV] Q42. What is the difference between Float.compare(a,b) and a>b style comparisons for NaN?

Float.compare(NaN, NaN) returns 0 (equal). NaN == NaN is false; NaN > x is false for all x. Float.compare uses a total ordering: negative zero < positive zero, and NaN is treated as greater than all values including POSITIVE_INFINITY. This consistent ordering is required by Comparable's contract (transitivity). Use Float.compare/Double.compare in Comparators, never subtraction or < / > for floating-point values.

[ADV] Q43. What happens with char arithmetic: char c = 'a'; c += 1; vs char c2 = c + 1?

c += 1 is fine — compound operators include an implicit narrowing cast to char. Equivalent to c = (char)(c + 1); result = 'b'. char c2 = c + 1 is a compile error — c+1 produces int (numeric promotion), and int cannot be assigned to char without an explicit cast. This asymmetry between += and simple + catches many developers.

[ADV] Q44. How can you use bit manipulation to determine if an int has the same sign as another?

XOR the sign bits: (a ^ b) >= 0 means same sign (sign bit XOR = 0 when both same). More precisely: boolean sameSign = ((a ^ b) >>> 31) == 0; The >>> 31 isolates the sign bit into bit position 0. Works correctly for all int values including MIN_VALUE. This is a micro-optimisation used in low-level libraries — always prefer readability (Math.signum) in application code.

[ADV] Q45. Explain how HashMap's hash() function improves on raw hashCode().

HashMap applies: hash = key.hashCode() ^ (h >>> 16). This XORs the high 16 bits with the low 16 bits (spreading high-bit information into the low bits). With small capacity (e.g. 16), only the lowest bits matter for indexing (hash & (cap-1)). Without spreading, poor hashCode implementations that differ only in high bits would collide in every bucket. The spread also partially compensates for hashCodes that differ only by constant multiples.

[ADV] Q46. What is the difference between Integer.MAX_VALUE and Long.MAX_VALUE in terms of bit representation?

Integer.MAX_VALUE = 0x7FFF_FFFF (32 bits, sign bit 0, all others 1 = 2^{31}-1). Long.MAX_VALUE = 0x7FFF_FFFF_FFFF_FFFFL (64 bits, 2■^{3}-1). Both use 2's complement. MIN_VALUE = MAX_VALUE + 1 (overflow). Integer.toBinaryString(Integer.MAX_VALUE) = 31 ones. Knowing these exact values matters for bounds checks, buffer sizing, and timestamp arithmetic.

Part 8 — Real-World & Production Scenarios

[ADV] Q47. A bug report says a HashMap stops finding keys after insertion. What could cause this?

Most likely cause: mutable field used in hashCode/equals was modified after the object was inserted into the HashMap. The object was inserted at bucket hashCode■; after mutation hashCode■ ≠ hashCode■; get() looks in the wrong bucket and returns null. The object is effectively lost. Fix: use only immutable fields (final fields) in hashCode/equals; or use records. Always document if a class is suitable as a map key.

[ADV] Q48. How would you safely convert between int and byte[] for network serialisation?

Use ByteBuffer or bit shifting: byte[] toBytes(int v) { return new byte[]{(byte)(v>>24),(byte)(v>>16),(byte)(v>>8),(byte)v}; } int fromBytes(byte[] b) { return (b[0]<<24)|(b[1]&0xFF)<<16|(b[2]&0xFF)<<8|(b[3]&0xFF); } Critical: b[1]&0xFF masks the sign extension that occurs when byte is widened to int. ByteBuffer.allocate(4).putInt(v).array() is cleaner. Specify ByteOrder explicitly (BIG_ENDIAN for network protocols, per RFC).

[ADV] Q49. Explain a production scenario where autoboxing caused a severe performance problem.

Real case pattern: a hot-path method accumulates scores: Map scores; scores.merge(key, 1L, Long::sum). Each merge unboxes the existing Long, adds 1 (creating a new Long), and boxes the result. Under high throughput, millions of Long objects are created per second, causing frequent minor GC pauses. Fix: switch to a primitive-friendly map (Trove TObjectLongHashMap, Eclipse Collections, or a direct long[] with custom key array), or use LongAdder per key for concurrent access.

[ADV] Q50. A timestamp comparison bug: two Date objects with the same millisecond return false from ==. What are the issues and fixes?

Issue 1: == compares references, not values. Two Date objects with same time are different heap objects → false. Fix: use date1.equals(date2) or date1.getTime()==date2.getTime(). Issue 2: java.util.Date is mutable — it can be changed after creation. Use java.time.Instant (immutable) for modern code. Issue 3: Date has resolution of milliseconds; Instant has nanosecond precision. Issue 4: Date's toString uses the system timezone which can mislead debugging. Always prefer java.time.* (Instant, LocalDate, ZonedDateTime).

[ADV] Q51. How do you avoid integer overflow when computing the midpoint of two ints for binary search?

WRONG: mid = (lo + hi) / 2 — can overflow if lo+hi > Integer.MAX_VALUE. CORRECT: mid = lo + (hi - lo) / 2 — subtraction is safe since hi >= lo. Alternative: mid = (lo + hi) >>> 1 — unsigned right shift avoids signed overflow issues. This was a real bug in Java's own Arrays.binarySearch for 20+ years. JDK was fixed in 2006.

[ADV] Q52. What is the -XX:AutoBoxCacheMax flag, and why might increasing it help or hurt performance?

It raises the upper bound of the Integer cache (default 127). Help: if your code heavily uses Integer values just above 127 (e.g., 128-256 for status codes/IDs), more objects are cached → fewer allocations → less GC. Hurt: the entire cache is pre-allocated at JVM startup from the old generation — too large a cache wastes heap and increases startup time. Measure before tuning. This is a niche flag; usually profile-guided Boxing elimination via escape analysis is more effective.

Exercises

Part A — Coding Challenges

A1. Write a method safeAdd(int a, int b) that returns an OptionalLong — containing the sum as a long if it fits in int, empty otherwise. No try/catch, no Math.addExact.

A2. Implement a method int reverseBytes(int n) using bit operations only (no String/array). Example: 0x12345678 → 0x78563412.

A3. Write a generic method > boolean isInRange(T value, T min, T max) that works for Integer, Double, String, etc. Handle null safely.

A4. Implement equals() and hashCode() for a 3D Point class (double x, y, z) that handles NaN coordinates correctly (two Points with NaN coordinate should be equal if and only if they are the same object, matching Double's behavior).

A5. Write a utility method parseIntSafe(String s, int defaultValue) that returns defaultValue for null, empty, or non-numeric input rather than throwing — without using exceptions for flow control.

Part B — Debug & Fix

B1 – Integer Cache Trap

```
Integer score1 = 127;
Integer score2 = 127;
if (score1 == score2) { reward(); } // Works by accident!
Integer score3 = 200;
Integer score4 = 200;
if (score3 == score4) { reward(); } // Silent bug — never rewards!
```

Fix: == compares references; works for 127 (cached) but fails for 200 (not cached). Fix: if (score3.equals(score4)) { reward(); }. Or use int primitives throughout if wrapper is not needed.

B2 – Overflow in Loop Bound

```
int count = Integer.MAX_VALUE;
for (int i = 0; i <= count; i++) { // INFINITE LOOP!
process(i);
}
```

Fix: i++ at MAX_VALUE wraps to MIN_VALUE; MIN_VALUE <= MAX_VALUE is always true → infinite loop. Fix: use long: for (long i = 0; i <= (long)count; i++). Or restructure the loop condition.

B3 – NaN Comparison

```
double speed = computeSpeed();
if (speed > 0) {
accelerate(); // NOT called when speed is NaN!
}
// NaN comparisons always return false
```

Fix: NaN > 0 is false, NaN <= 0 is also false — NaN fails all comparisons. Fix: if (!Double.isNaN(speed) && speed > 0) { accelerate(); }

B4 – Mutable Key in HashMap

```
class Range {
int start, end; // mutable fields
// equals/hashCode based on start,end
}
Map<Range, String> map = new HashMap<>();
Range r = new Range(1, 10);
map.put(r, "hello");
r.start = 99; // mutate after insertion!
System.out.println(map.get(r)); // null — lost!
```

Fix: Mutating r.start changes its hashCode → different bucket → get() can't find it. Fix: make Range immutable (final fields, no setters). Use a Java record: record Range(int start, int end){}.

Part C — MCQ / Predict the Output

MC1. What is the output? byte b = (byte) 200; System.out.println(b);

A) 200
B) -56
C) 0
D) Compile error

Answer: B — 200 = 0xC8 = 1100_1000 in 8 bits = -56 in signed 2's complement (high bit set).

MC2. What is the output? System.out.println(0.1f == 0.1);

A) true
B) false
C) Compile error
D) Runtime exception

Answer: B — 0.1f is 32-bit float; 0.1 is 64-bit double. float is widened to double for comparison, but their bit patterns differ → false.

MC3. Which statement correctly creates a long literal of 1 billion?

A) long x = 1000000000;
B) long x = 1_000_000_000L;
C) long x = 1000000000L;
D) Both B and C

Answer: D — Both B and C are valid. A compiles too (int literal fits in long), but 10 billion would need L.

MC4. What is the output? char c='Z'; c += 1; System.out.println(c);

A) Compile error
B) 91
C) [
D) Z1

Answer: C — 'Z'=90; +=1 → 91; (char)91 = '[' (ASCII). += includes implicit (char) cast, so no error.

MC5. What is printed? Integer x = null; System.out.println(x == null ? 0 : x);

A) NPE
B) null
C) 0
D) Compile error

Answer: C — Ternary evaluates x==null (true), returns 0 (int literal). No unboxing of null occurs. Safe.

MC6. What does Integer.bitCount(255) return?

A) 8

B) 7

C) 255

D) 1

Answer: A — 255 = 0xFF = 1111_1111 in binary. 8 bits set. bitCount counts the number of 1-bits.

Quick Reference Cheat Sheet

Topic	Key Points to Remember
8 Primitives	byte(8),short(16),int(32),long(64),float(32 IEEE),double(64 IEEE),char(16 unsigned),boolean(1*)
Default values	Numeric=0, char=\u0000, boolean=false, reference=null. Local vars: MUST initialise explicitly
Widening order	byte→short→int→long→float→double; char→int. Long→float may lose precision!
Narrowing cast	Keeps low-order bits: (byte)300=44; truncates toward zero: (int)3.9=3
Numeric promotion	byte/short/char + any = int result. Mixed with long→long, float→float, double→double
Compound ops	b+=5 includes implicit (byte)cast. Simple b=b+5 is compile error if b is byte/char
Integer overflow	Silently wraps. MAX_VALUE+1=MIN_VALUE. Use Math.addExact() for checked arithmetic
Integer cache	[-128,127] cached by Byte/Short/Integer/Long; [0,127] for Character; Boolean singletons
== on wrappers	== compares references → use .equals() always. > < >= <= always unbox (NullPointerException risk)
NaN rules	NaN != NaN (always false). Use Double.isNaN(). Double.compare() treats NaN==NaN
0.0/0.0 vs 0/0	Float/double: returns NaN (no exception). Integer: ArithmeticException: / by zero
Float precision	float~7 sig digits, double~15. Never use == for calculated floats. Use epsilon or BigDecimal
String pool	Literals interned; new String() not. intern() adds to pool. Pool in Metaspace (Java 8+)
equals contract	Reflexive, Symmetric, Transitive, Consistent, Non-null. Override hashCode with equals!
Pass-by-value	Always. Object: reference copy (can mutate; can't reassign caller's var). No out/ref params
Bit ops	& | ^ ~ << >> >>>. >>> fills with 0 (unsigned). (n&(n-1))==0 tests power-of-2
Midpoint bug	lo+(hi-lo)/2 or (lo+hi)>>>1 — NOT (lo+hi)/2 which overflows
BigDecimal	Use for money. Construct from String: new BigDecimal("0.1"). Use setScale+RoundingMode

End of Sub-Chapter 1.1 — Java Primitives & Type System

Next: Sub-Chapter 1.2 — Collections Framework Overview & List Implementations